



ปริญญานิพนธ์

INTELLIGENT SERVER OPERATIONS PLATFORM

ระบบปฏิบัติการและบริหารจัดการเครื่องแม่ข่าย

วัชรปกรณ์ มีชัย

ปรัชญาพงษ์ คุณมี

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต

สาขาวิชา วิศวกรรมคอมพิวเตอร์

มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน วิทยาเขตสกลนคร

คณะอุตสาหกรรมและเทคโนโลยี มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน

วิทยาเขตสกลนคร พ.ศ. 2569

INTELLIGENT SERVER OPERATIONS PLATFORM

ระบบปฏิบัติการและบริหารจัดการเครื่องแม่ข่าย

ปริญญานิพนธ์

จัดทำโดย

วัชรปกรณ มีชัย

ปรัชญาพงษ์ คุณมี

อาจารย์ที่ปรึกษา: อาจารย์อัษฎาฐ เพชรพรรณ

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต

สาขาวิชา วิศวกรรมคอมพิวเตอร์

ปีที่ปริญญานิพนธ์สำเร็จ พ.ศ. 2569

บทคัดย่อ

ปริญญานิพนธ์นี้พัฒนาโปรแกรมสำหรับผู้ดูแลระบบติดตามทรัพยากรของเครื่องคอมพิวเตอร์และจัดการ Docker container จากจุดเดียว โปรแกรมอ่านการใช้หน่วยประมวลผล หน่วยความจำ และพื้นที่จัดเก็บของเครื่องที่เปิดโปรแกรม พร้อมบันทึกค่าลงไฟล์ CSV เพื่อย้ายโอนหลัง ส่วน container สามารถนำเข้ารายการบริการจากไฟล์ Docker Compose แล้วตรวจสอบสถานะ เรียกดู log ส่งเริ่ม หยุด หรือเริ่มใหม่ และกำหนดเพดาน CPU กับ RAM ได้

การตรวจจับสถานะใช้ค่าเฉลี่ยเคลื่อนที่แบบเอ็กซ์โพเนนเชียล (EMA) กับเกณฑ์เข้าและออกจากสถานะผิดปกติที่ต่างกัน ระบบไม่ต้องฝึกโมเดลจากชุดข้อมูล การแจ้งเตือนและการดำเนินการแก้ไขบางอย่างทำงานตามเงื่อนไขที่กำหนด ผู้ใช้ยังเข้าถึงข้อมูลและจัดการ container ผ่านเว็บแดชบอร์ดที่มีหน้าเข้าสู่ระบบได้ และส่งลิงก์แดชบอร์ดทางอีเมลได้เมื่อกำหนดค่า SMTP ส่วนการสำรองข้อมูลสร้างไฟล์ ZIP จากไฟล์เดอร์แหล่งข้อมูลที่โปรแกรมกำหนดหรือผู้ใช้งานเข้า ระบบมีตัวเลือกปรับเพดาน RAM ของ container อัตโนมัติ ซึ่งผู้ใช้ต้องเปิดใช้เป็นราย container

หลักฐานการทดสอบที่มีประกอบด้วยรายงานทดสอบเชิงหน้าที่ 10 รายการซึ่งส่งมาพร้อมโครงการ และผล self-test ของไฟล์ EXE บน Windows จำนวน 7 รายการเมื่อวันที่ 23 กันยายน 2569 รายงานแรกระบุว่าผ่านกรณีทดสอบตามรายการ ส่วน self-test ยืนยันการโหลดทรัพยากรและการเปิดโปรแกรมแบบ one-file ได้ ผลดังกล่าวยังไม่ใช่การวัดประสิทธิภาพระยะยาวหรือการทดสอบบนเครื่องแม่ข่ายหลายสภาพแวดล้อม และยังไม่มีข้อมูลสำหรับคำนวณเวลาตรวจจับหรือการลด downtime

คำสำคัญ: เครื่องแม่ข่าย, Docker, การติดตามทรัพยากร, การตรวจจับความผิดปกติ, การบริหารระบบ

Abstract

This project developed an application to help administrators monitor a computer host and manage Docker containers from one interface. It collects CPU, memory, and disk usage from the computer running the application and stores metric samples in CSV files. Docker Compose files can be imported to identify services and containers for monitoring. The application supports container status checks, log viewing, start, stop, restart, and CPU and memory limit updates.

Resource conditions are evaluated using an exponential moving average (EMA) and separate thresholds for entering and leaving an abnormal state. The method is rule-based and does not require a trained machine-learning model. The application also provides in-app alerts, selected automated remediation actions, an authenticated web dashboard, SMTP email delivery for dashboard links, and ZIP backups of configured project folders. An optional per-container RAM limit manager adjusts limits only when enabled by the user.

Available test evidence consists of a submitted report listing ten functional test cases as passed and a Windows executable self-test that passed seven runtime and packaging checks on 23 September 2026. These results cover the reported cases and executable startup checks; they do not establish long-term stability, performance across server environments, detection latency, or reduced downtime.

Keywords: server monitoring, Docker, resource monitoring, anomaly detection, system administration

กิตติกรรมประกาศ

คณะผู้จัดทำขอขอบพระคุณอาจารย์อัษฎาวุธ เพชรพรรณ อาจารย์ที่ปรึกษาปริญญานิพนธ์
ที่ให้คำแนะนำและข้อเสนอแนะระหว่างการดำเนินงาน

ขอขอบพระคุณคณาจารย์และบุคลากรของสาขาวิชาและคณะอุตสาหกรรมและเทคโนโลยี
มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน วิทยาเขตสกลนคร

ที่สนับสนุนการศึกษาและการจัดทำโครงการ รวมถึงครอบครัวและเพื่อนที่ให้ความสนใจตลอดมา

คณะผู้จัดทำขอมอบคุณค่าและประโยชน์จากปริญญานิพนธ์ฉบับนี้แก่ผู้ที่มีส่วนช่วยเหลือทุกทำ
น หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขอน้อมรับคำแนะนำเพื่อนำไปปรับปรุงต่อไป

คณะผู้จัดทำ

สารบัญ

บทคัดย่อ	3
Abstract	4
กิตติกรรมประกาศ	5
บทที่ 1 บทนำ	12
1.1 ความเป็นมาและความสำคัญ	12
1.2 ลักษณะปัญหาและความต้องการ	12
1.3 วัตถุประสงค์	13
1.4 ขอบเขต	14
1.5 ประโยชน์ที่คาดว่าจะได้รับ	14
1.6 คำศัพท์ที่ใช้ในรายงาน	15
1.7 คำถามและตัวชี้วัดสำหรับการประเมิน	15
บทที่ 2 แนวคิดและเทคโนโลยีที่เกี่ยวข้อง	17
2.1 การติดตามทรัพยากรระบบ	17
2.2 การปรับค่าให้เรียบด้วย EMA	17
2.3 Hysteresis และเกณฑ์สถานะ	17
2.4 การตรวจจับด้วยกฎและความหมายของ risk score	17
2.5 การควบคุมทรัพยากร container	18
2.6 Docker Compose และ YAML	18
2.7 เว็บแอปพลิเคชันด้วย Flask	19
2.8 การเข้าถึงผ่าน Cloudflare Tunnel	19
2.9 CSV และการสำรองข้อมูล	19
2.10 ความปลอดภัยและขอบเขตการใช้งาน	19
2.11 เปรียบเทียบเครื่องมือและขอบเขตของโครงการ	20
2.12 โครงสร้างพื้นฐานเครื่องแม่ข่าย	20
2.13 การบริหารงานปฏิบัติการ IT	21
2.14 AIOps ในฐานะแนวคิดอ้างอิง	21
2.15 แนวทางตรวจจับความผิดปกติที่เกี่ยวข้อง	21

2.16 การวิเคราะห์หอนุกรมเวลาและการบำรุงรักษาเชิงพยากรณ์	22
2.17 มุมมองการแผ่รังสีบริการ	22
2.18 ตัวชี้วัดสำหรับการประเมินการตรวจจับ	22
2.19 ขอบเขตการประยุกต์ใช้แนวคิดในระบบ	23
2.20 สรุป	23
บทที่ 3 การออกแบบและพัฒนา	24
3.1 ภาพรวมการทำงาน	24
3.2 ลำดับการทำงานหลัก	24
3.3 รูปแบบข้อมูลและไฟล์	25
3.4 การจัดการสถานะและข้อผิดพลาด	26
3.5 การเก็บและประเมิน metrics	26
3.6 การจัดการ Docker	27
3.7 เว็บแดชบอร์ดและการเชื่อมต่อ	27
3.8 การแจ้งเตือนและการดำเนินการอัตโนมัติ	27
3.9 การสำรองข้อมูล	28
3.10 เครื่องมือและสภาพแวดล้อมที่ยืนยันได้	28
3.11 การสร้างไฟล์ส่งมอบ	28
3.12 วิธีประเมินผล	28
บทที่ 4 ผลการพัฒนาและการทดสอบ	29
4.1 ผลการพัฒนา	29
4.2 หน้าจอของโปรแกรม	29
4.3 รายงานผลทดสอบเชิงหน้าที่	30
4.4 ผล self-test ของ Windows EXE	31
4.5 อภิปรายผล	32
4.6 ข้อจำกัดของผลทดสอบ	32
บทที่ 5 สรุปผลและข้อเสนอแนะ	33
5.1 สรุปผล	33
5.2 ข้อจำกัดของระบบ	33
5.3 ข้อเสนอแนะสำหรับการพัฒนาต่อ	33

5.4 งานที่ควรทำก่อนส่งฉบับสมบูรณ์	34
บรรณานุกรม	35
ภาคผนวก ก	38
แนวทางการใช้งาน	39
1. เตรียมเครื่อง	39
2. เปิดโปรแกรม	39
3. เพิ่มบริการ	39
4. จัดการ container	39
5. ใช้เว็บแดชบอร์ด	39
6. สำรองข้อมูล	39
ภาคผนวก ข	40
ภาคผนวก ค	41
ประวัติผู้เขียน	42

สารบัญรูป

รูปที่ 4.1 หน้าหลักสำหรับติดตามทรัพยากร

รูปที่ 4.2 หน้าประวัติและกราฟการใช้ทรัพยากร

รูปที่ 4.3 หน้า Server Dashboard

รูปที่ 4.4 หน้า Server Manager

รูปที่ 4.5 หน้าต่าง Terminal

รูปที่ 4.6 หน้า Action Log

สารบัญตาราง

ตารางที่ 1.1 ความต้องการของระบบ

ตารางที่ 2.1 เปรียบเทียบขอบเขตเครื่องมือ

ตารางที่ 3.1 หน้าที่ของโมดูล

ตารางที่ 3.2 ลำดับการทำงานหลัก

ตารางที่ 3.3 รูปแบบไฟล์ข้อมูล

ตารางที่ 4.1 รายการทดสอบเชิงหน้าที่

ตารางที่ 4.2 ผลทดสอบที่รายงาน

บทที่ 1 บทนำ

1.1 ความเป็นมาและความสำคัญ

ระบบสารสนเทศของสถาบันการศึกษาและองค์กรทั่วไปอาศัยเครื่องมือข่ายรองรับงานหลายประเภท เช่น เว็บไซต์ ระบบข้อมูล และบริการภายใน ผู้ดูแลจึงต้องติดตามทั้งภาวะทรัพยากรและสถานะบริการ หากมีหลายบริการหรือใช้ container การตรวจที่ละหน้าจอและสลับคำสั่งไปมาอาจทำให้มองภาพรวมได้ยากขึ้น ข้อความนี้เป็นบริบทของปัญหา ไม่ได้หมายความว่าโครงการถูกติดตั้งในศูนย์ข้อมูลของมหาวิทยาลัยหรือมีข้อมูลใช้งานจริงจากระบบทะเบียน/ระบบสอบ

โครงการนี้เริ่มจากความต้องการรวมงานพื้นฐานดังกล่าวไว้ในโปรแกรมเดียว ผู้ใช้ดูค่าทรัพยากรของเครื่อง จัดการ container ที่เลือกจากไฟล์ Docker Compose และเปิดหน้าเว็บเพื่อดูสถานะหรือสั่งงานบางอย่างได้ การแจ้งเตือนอาศัยเกณฑ์ที่กำหนดไว้ล่วงหน้า จึงตอบสนองได้โดยไม่ต้องเตรียมข้อมูลฝึกโมเดล แต่ยังคงเลือกเกณฑ์ให้เหมาะกับลักษณะงานจริง

ชื่อโครงการใช้คำว่า Intelligent Server Operations Platform ตามชื่อระบบที่พัฒนา กลไกตรวจจับในรุ่นที่ตรวจสอบเป็นการคำนวณ EMA และกฎ threshold ไม่ใช่โมเดล machine learning ที่ฝึกจากข้อมูล รายงานฉบับนี้จึงอธิบายวิธีทำงานตามโค้ดและหลักฐานที่มี และไม่ถือว่าระบบหาสาเหตุของปัญหาหรือรับประกันการป้องกัน downtime

1.2 ลักษณะปัญหาและความต้องการ

ปัญหาที่โครงการเลือกแก้เป็นงานติดตามและควบคุมขั้นพื้นฐาน ซึ่งผู้ดูแลมักทำซ้ำหลายครั้ง ได้แก่ ตรวจการใช้ทรัพยากร ดูว่า container ยังทำงานหรือไม่ และเรียกดู log เมื่อบริการมีปัญหา หากแต่ละงานอยู่คนละเครื่องมือ ผู้ใช้ต้องสลับหน้าจอและจดจำชื่อบริการหรือคำสั่งเอง โครงการจึงรวบรวมข้อมูลที่จำเป็นต่อการตรวจเบื้องต้นไว้ในแอปเดียว

ผู้ใช้หลักคือผู้ดูแลที่มีสิทธิ์เข้าถึงเครื่องและ Docker อยู่แล้ว ระบบไม่ทำหน้าที่แทนระบบเฝ้าระวังองค์กร และไม่จัดการเครื่องปลายทางโดยไม่ผ่าน Docker หรือวิธีเชื่อมต่อที่โครงการรองรับ การติดตามของ host เป็นค่าจากเครื่องที่รันแอป ส่วนการติดตาม container ทำเฉพาะรายการที่ผู้ใช้เลือกนำเข้า

รหัส	ความต้องการ	พฤติกรรมที่คาดหวัง
F01	ดูทรัพยากรเครื่อง	แสดง CPU, RAM และ disk ของเครื่องที่แอปกำลังทำงาน

รหัส	ความต้องการ	พฤติกรรมที่คาดหวัง
F02	ดูประวัติ	อ่านค่าล่าสุดจาก CSV และนำไปแสดงกราฟ
F03	ตรวจสอบสถานะ	ปรับค่า EMA และเปลี่ยนสถานะเมื่อเข้า/ออก threshold
F04	จัดการ Docker	แสดงรายการที่เลือก และรองรับ start/stop/restart/logs/exec
F05	กำหนด resource limit	บันทึก CPU/RAM limit และส่งคำสั่งปรับให้ Docker
F06	ดูผ่านเว็บ	ต้อง login ก่อนเข้าหน้า dashboard และ API สำหรับจัดการ
F07	สำรองข้อมูล	สร้าง ZIP ตามรายการแหล่งข้อมูลและปลายทางที่ตั้งไว้
F08	ปรับ RAM อัตโนมัติ	ทำงานเฉพาะ container ที่ผู้ใช้เปิดตัวเล็ กไว้

ข้อกำหนดด้านการใช้งานคือหน้าจอลงมือเข้าถึงข้อมูลหลักได้โดยไม่ต้องจำคำสั่ง Docker สำหรับงานทั่วไป และข้อมูลการตั้งค่าต้องยังอยู่หลังปิดโปรแกรม ส่วนข้อกำหนดด้านความปลอดภัยและเสถียรภาพยังต้องประเมินเพิ่มเติม โดยเฉพาะสิทธิ์ Docker, การเปิด tunnel, การเก็บรหัสผ่าน SMTP และการยืนยันผลการกู้คืน backup

1.3 วัตถุประสงค์

- พัฒนาโปรแกรมเดสก์ท็อปสำหรับติดตาม CPU หน่วยความจำ และพื้นที่จัดเก็บของเครื่องที่ใช้งาน
- พัฒนากลไกประเมินสถานะทรัพยากรด้วย EMA และเกณฑ์เข้าออกสถานะที่กำหนด
- รวมการจัดการ Docker container และการกำหนดเพดาน CPU/RAM ไว้ในส่วนติดต่อเดียว พร้อมตัวเลือกปรับ RAM อัตโนมัติ
- จัดทำเว็บแดชบอร์ดที่มีการเข้าสู่ระบบและส่งลิงก์ผ่านอีเมลสำหรับติดตามสถานะและสั่งงาน container
- จัดทำกลไกสำรองไฟล์โครงการที่เลือกในรูปแบบ ZIP

1.4 ขอบเขต

- พัฒนาโปรแกรมด้วย Python และส่วนติดต่อเดสก์ท็อป CustomTkinter โดยไฟล์ส่งมอบที่บันทึกไว้เป็น Windows EXE แบบ one-file
- อ่านค่าทรัพยากรของเครื่องที่โปรแกรมทำงานอยู่ด้วย psutil; พื้นที่ที่ติดสื่อกับพาร์ติชันระบบหลัก (C:\ บน Windows หรือ / บน Linux)
- ใช้ Docker CLI เพื่ออ่านสถานะ สั่งเริ่ม/หยุด/เริ่มใหม่ ดู log และส่งคำสั่งเข้า container ที่เลือก
- นำเข้า YAML ที่เป็น Docker Compose เพื่อระบุบริการและ container; ไฟล์ Compose ของแอปเองใช้เปิดบริการเว็บและ tunnel เมื่อกำหนดไว้
- ใช้ EMA alpha 0.3 และเกณฑ์ anomaly 90% สำหรับเข้าและ 80% สำหรับออก โดยอิงค่าสูงสุดของ CPU, RAM และ disk ที่ผ่าน EMA
- ให้เว็บแดชบอร์ดรับการเข้าสู่ระบบ และป้องกันเส้นทาง API สำหรับการจัดการด้วยการตรวจสอบ session
- รองรับการส่งลิงก์ dashboard ผ่าน SMTP เมื่อผู้ใช้กำหนดอีเมลผู้ส่งและผู้รับ
- มีระบบปรับ RAM ของ container ที่ผู้ใช้เปิดสวิตช์ราย container; ใช้ EMA, headroom และเพดานส่วนกลางในการคำนวณ
- สำรองไฟล์/โฟลเดอร์ที่ระบบเลือกเป็น ZIP; ไม่สำรอง host ทั้งเครื่องหรือ Docker volumes โดยอัตโนมัติ

1.5 ประโยชน์ที่คาดว่าจะได้รับ

ผู้ดูแลมีจุดเริ่มต้นสำหรับดูสถานะทรัพยากรและจัดการ container ลดการสลับระหว่างหน้าจอ และใช้ประวัติ CSV ตรวจสอบแนวโน้มย้อนหลังได้ การนำเกณฑ์และการดำเนินการอัตโนมัติมารวมไว้ยังช่วยให้เห็นแนวทางการสร้างเครื่องมือดูแลระบบขนาดเล็ก อย่างไรก็ตาม ประโยชน์ด้านเวลาทำงานหรือ downtime ต้องประเมินจากการใช้งานจริงเพิ่มเติม

1.6 คำศัพท์ที่ใช้ในรายงาน

คำ	ความหมาย
Container	หน่วยสำหรับรันแอปพลิเคชันที่จัดการด้วย Docker
Docker Compose	รูปแบบไฟล์ YAML สำหรับประกาศบริการและค่าที่เกี่ยวข้องกับ Docker Compose
EMA	ค่าเฉลี่ยที่ให้น้ำหนักข้อมูลล่าสุดมากกว่าข้อมูลก่อนหน้า
Hysteresis	การใช้เกณฑ์คนละค่าตอนเข้าสถานะและออกจากสถานะ เพื่อลดการเปลี่ยนสถานะถี่
Remediation	การดำเนินการแก้ไขตามเงื่อนไขที่โปรแกรมกำหนด
One-file executable	ไฟล์ EXE ที่บรรจุทรัพยากรที่จำเป็นเพื่อเปิดโปรแกรมจากไฟล์เดียว

1.7 คำถามและตัวชี้วัดสำหรับการประเมิน

การประเมินระบบควรแยกคำถามเชิงหน้าที่ออกจากคำถามเชิงประสิทธิภาพ คำถามเชิงหน้าที่ตรวจว่าโปรแกรมอ่าน metric ได้ เปลี่ยนสถานะตามเกณฑ์ สั่งงาน Docker และเปิดเว็บได้หรือไม่ ส่วนคำถามเชิงประสิทธิภาพต้องใช้ข้อมูลหลายรอบและสภาพแวดล้อมที่ระบุชัดเจน เช่น เวลาจากเริ่มเหตุถึงแจ้งเตือน อัตราการตรวจพบ และจำนวน false alarm

คำถาม	ตัวชี้วัดที่เหมาะสม	สถานะหลักฐาน
ค่าที่แสดงอ่านได้หรือไม่	เปรียบเทียบกับเครื่องมือ OS พร้อม timestamp	มีรายงานว่ายืนยันแล้ว แต่ไม่มีตัวเลขแนบ
สถานะ anomaly เปลี่ยนตามกฎหรือไม่	trace ของค่าเข้า/EMA/สถานะ	รายงานเดิมระบุผ่าน ไม่มี trace
ระบบตรวจพบเหตุได้ครบเพียงใด	$\text{Recall} = \text{TP}/(\text{TP}+\text{FN})$	ยังไม่มี labelled event set
การแจ้งเตือนเกินจริงหรือไม่	$\text{Precision} = \text{TP}/(\text{TP}+\text{FP})$	ยังไม่มีผลคำนวณ
การกู้คืนเร็วขึ้นหรือไม่	MTTR และ baseline ก่อน/หลัง	ไม่มีข้อมูลเปรียบเทียบ
โปรแกรมเสถียรแค่ไหน	uptime, error count, resource overhead	ยังไม่มี long-run report

ตัวชี้วัดในตารางเป็นกรอบสำหรับเก็บหลักฐานในอนาคต ไม่ใช่ผลของโครงการในปัจจุบัน การตั้งเป้าหมายเชิงตัวเลข เช่น ลด MTTR หรือกำหนด availability ต้องระบุวิธีวัด ช่วงเวลา และ

baseline ล้วงหน้า และยังสามารถสรุปไม่ได้หากไม่มีข้อมูลทดลองรองรับ

บทที่ 2 แนวคิดและเทคโนโลยีที่เกี่ยวข้อง

2.1 การติดตามทรัพยากรระบบ

การติดตามทรัพยากรช่วยให้เห็นภาระงานของเครื่องในช่วงเวลาหนึ่ง โครงการนี้อ่านค่า CPU, RAM และพื้นที่จัดเก็บผ่าน psutil ซึ่งเป็นไลบรารี Python สำหรับอ่านข้อมูลระบบและ process หลายระบบปฏิบัติการ [1] ค่าที่แสดงจึงเป็นค่าของเครื่องที่โปรแกรมทำงานอยู่ ส่วนรายการ container ใช้ Docker CLI แยกต่างหาก

2.2 การปรับค่าให้เรียบด้วย EMA

ค่าการใช้ CPU อาจเปลี่ยนแปลงระหว่างการอ่านแต่ละครั้ง ระบบจึงคำนวณ EMA เพื่อไม่ให้เกิดการตัดสินใจขึ้นกับค่ากระชากเพียงครั้งเดียว สมการที่ใช้คือ $E_t = x_t + (1 - \alpha)E_{t-1}$ โดย x_t คือค่าที่อ่านได้ในรอบปัจจุบัน E_{t-1} คือค่าเฉลี่ยรอบก่อน และ α คือค่าน้ำหนักของข้อมูลใหม่ ในโค้ดกำหนด $\alpha = 0.3$

ค่า α ที่ต่ำทำให้ผลเปลี่ยนช้าลง ส่วนค่าที่สูงทำให้ผลตามข้อมูลใหม่เร็วขึ้น ในโครงการนี้ค่าเป็นค่าคงที่และยังไม่มีผลทดลองเปรียบเทียบค่า α หลายระดับ จึงควรมองเป็นค่าตั้งต้นของต้นแบบ

2.3 Hysteresis และเกณฑ์สถานะ

ระบบเปลี่ยนเป็นสถานะผิดปกติเมื่อค่า EMA สูงสุดในสามตัวชี้วัดมากกว่า 90% และกลับเป็นปกติเมื่อค่าดังกล่าวต่ำกว่า 80% ช่วงระหว่าง 80-90% จึงคงสถานะเดิมไว้ กลไกนี้ลดการสลับสถานะไปมาบริเวณเส้นแบ่ง แต่ไม่ได้ยืนยันว่าค่าที่สูงเกิดจากความขัดข้อง เพราะอาจเป็นภาระงานปกติของบริการ

2.4 การตรวจจับด้วยกฎและความหมายของ risk score

ในระบบเผื่อระวัง ค่าที่เกินเกณฑ์เป็นสัญญาณให้ผู้ดูแลตรวจสอบต่อ ไม่ใช่คำวินิจฉัยสาเหตุ การใช้เกณฑ์แบบกำหนดทำให้เส้นทางการตัดสินใจอธิบายได้ง่ายกว่าระบบที่ต้องฝึกโมเดล แต่ threshold เดียวกันอาจให้ผลต่างกันเมื่อเปลี่ยนชนิดบริการหรือขนาดเครื่อง

ให้ x_t แทนค่าที่อ่านในเวลา t และ E_t แทนค่า EMA จะได้ $E_t = 0.3x_t + 0.7E_{t-1}$ สำหรับ CPU, RAM และ disk แยกกัน ในรอบแรกใช้ค่าที่อ่านเป็นค่าเริ่มต้น จากนั้นหาค่าสูงสุดของ EMA ทั้งสามเป็น risk score ที่แสดงในโปรแกรม

ตัวเลขนี้จึงเท่ากับตัวชี้วัดที่ผ่านการปรับเรียบซึ่งสูงสุดในขณะนั้นไม่ใช่ความน่าจะเป็นที่ระบบจะล้มเหลว

Hysteresis ใช้เกณฑ์ 90% ตอนเปลี่ยนจากปกติเป็นผิดปกติ และ 80% ตอนกลับเป็นปกติ ตัวอย่างเช่น เมื่อสถานะผิดปกติแล้ว ค่า 85% จะยังคงสถานะเดิม จนค่าลดต่ำกว่า 80% ข้อดีคือไม่เปลี่ยนสถานะทุกครั้งที่ค่าแกว่งใกล้ 90% แต่ต้องทดสอบกับข้อมูลจริงเพื่อปรับเกณฑ์ให้เหมาะกับงาน

2.5 การควบคุมทรัพยากร container

Docker สามารถจำกัด CPU และหน่วยความจำของ container ได้ [2] โครงการเปิดให้ผู้ตั้งค่า CPU เป็นเปอร์เซ็นต์และ RAM เป็นเมกะไบต์ ค่า CPU 100% หมายความว่าไม่จำกัด quota ในเครื่องมือ ส่วนการปรับค่าจะอาศัย Docker update และตรวจ memory limit ที่ Docker รายงานกลับ การตั้งค่าเหล่านี้อาจถูกจำกัดโดยระบบปฏิบัติการหรือความสามารถของ Docker Engine

ระบบปรับ RAM อัตโนมัติเป็นทางเลือกที่ปิดไว้เป็นค่าเริ่มต้นและเปิดได้ต่อ container กลไกอ่าน RAM ที่ใช้จริง ทำ EMA ด้วย alpha 0.4 คูณ headroom 1.4 บัดขึ้นเป็นช่วงละ 64 MB และจำกัดผลด้วยค่า ceiling ที่ผู้ตั้งไว้ (ขั้นต่ำ 128 MB) จะเรียกปรับเมื่อค่าที่เสนอแตกต่างจากค่าเดิมอย่างน้อย 15% และประเมินเป็นรอบประมาณ 20 วินาที แนวทางนี้พยายามลดการตั้ง limit ต่ำกว่าการใช้จริง แต่ไม่ใช่การทำนายที่รับประกันว่าจะไม่เกิด out-of-memory

2.6 Docker Compose และ YAML

Docker Compose ใช้ไฟล์ YAML เพื่ออธิบายบริการหลายรายการและความสัมพันธ์สำหรับการเปิดใช้งานร่วมกัน โครงการอ่านไฟล์ที่ผู้ใช้เลือกเพื่อแสดงบริการและนำ container ที่เกี่ยวข้องเข้าสู่รายการดูแล ส่วน docker-compose.yml ที่ฝังมากับแอปเป็นคอนฟิกและใช้เตรียมเว็บแดชบอร์ดหรือ tunnel การแยกสองบทบาทนี้ช่วยไม่ให้เข้าใจผิดว่า YAML ทุกไฟล์ถูกใช้เป็น config สำหรับกฎ remediation

2.7 เว็บแอปพลิเคชันด้วย Flask

Flask เป็นเว็บ framework สำหรับ Python ที่เชื่อม URL กับฟังก์ชันของแอปและสามารถเรนเดอร์ template [3] โครงการใช้ Flask ให้บริการหน้า login, dashboard และ API สำหรับอ่านสถานะหรือส่งคำสั่งไปยัง Docker ส่วนหน้าจอบริการเรียก API เป็นระยะเพื่อปรับข้อมูลบนหน้า

2.8 การเข้าถึงผ่าน Cloudflare Tunnel

ระบบสามารถใช้ Cloudflare Quick Tunnel เพื่อสร้าง URL สาธารณะสำหรับทดลองเข้าถึงเว็บจากเครือข่ายภายนอก โดย URL ถูกสร้างใหม่เมื่อเชื่อมต่อ คุณมี Cloudflare ระบุว่า Quick Tunnel มีไว้สำหรับการทดสอบและพัฒนา และไม่รับประกันการให้บริการระดับ production [4] การเปิดหน้าเว็บสู่เครือข่ายภายนอกจึงต้องพิจารณาความปลอดภัยและรูปแบบ tunnel ให้เหมาะกับการใช้งาน

2.9 CSV และการสำรองข้อมูล

ค่าทรัพยากรถูกเขียนต่อท้ายไฟล์ CSV พร้อมเวลา CPU, RAM และ disk เพื่อให้เปิดดูหรือใช้สร้างกราฟย้อนหลังได้ ส่วนการสำรองข้อมูลสร้าง ZIP จากรายการแหล่งข้อมูลที่กำหนด ตั้งค่าเริ่มทำงานทันทีและวนทุกหนึ่งชั่วโมงเมื่อเปิดใช้งาน ระบบเก็บไฟล์สำรองชุดล่าสุด โดยสร้างไฟล์ชั่วคราวก่อนแทนที่ไฟล์เดิม เพื่อลดโอกาสที่ไฟล์เดิมหายหากการสร้างชุดใหม่ผิดพลาด การทำงานนี้ยังไม่ทดแทนการสำรอง volume หรือการทดสอบกู้คืน

2.10 ความปลอดภัยและขอบเขตการใช้งาน

การเปิด endpoint สำหรับสั่ง Docker ทำให้บัญชีเว็บมีอำนาจต่อบริการที่กำลังทำงาน จึงควรใช้รหัสผ่านที่คาดเดายาก จำกัดการเข้าถึงเครือข่าย และทบทวนการใช้ HTTPS เมื่อเปิดจากภายนอก เอกสาร Flask แยกเวิร์กโฟลเดอร์พัฒนาออกจากการ deploy จริง จึงไม่ควรนำผลการทดสอบ Flask test client ไปตีความเป็นการรับรองความปลอดภัยของการใช้งานจริง [3]

Quick Tunnel สร้าง URL สุ่มเพื่อใช้เข้าถึงบริการ แต่เอกสาร Cloudflare ระบุว่าออกแบบสำหรับทดสอบและพัฒนา หากต้องเปิดใช้เป็นบริการจริงควรพิจารณา tunnel

ที่จัดการถาวรและกำหนดนโยบายการเข้าถึง [4] รหัสผ่าน SMTP และข้อมูลบัญชีควรได้รับการปกป้อง การสำรอง ZIP ที่มีไฟล์ตั้งค่าก็ควรจัดเก็บในตำแหน่งที่ได้รับอนุญาต

2.11 เปรียบเทียบเครื่องมือและขอบเขตของโครงการ

งาน	เครื่องมือทั่วไป	ขอบเขตของโครงการ
ตรวจ host	เครื่องมือระบบปฏิบัติการ	แสดง CPU/RAM/disk ในหน้าจอเดียวและบันทึก CSV
จัดการ container	Docker CLI / Docker Desktop	เรียกคำสั่งหลักจากแอปและเว็บ
ดูข้อมูลย้อนหลัง	เปิดไฟล์/กราฟภายนอก	อ่าน CSV ล่าสุดและแสดงกราฟในโปรแกรม
เข้าถึงนอก LAN	ตั้งค่า reverse proxy/tunnel เอง	เชื่อม Quick Tunnel ตาม Compose ที่เตรียมไว้
แจ้งเตือน	เครื่องมือ monitoring เฉพาะทาง	popup และข้อความในโปรแกรมตามกฎหมายที่กำหนด

ตารางนี้เปรียบเทียบขอบเขตหน้าที่ ไม่ใช้การทดสอบประสิทธิภาพเทียบผลิตภัณฑ์อื่น โครงการมุ่งรวมงานพื้นฐานสำหรับผู้ดูแลเครื่องที่ใช้ Docker มากกว่าทดแทนแพลตฟอร์ม monitoring ระดับองค์กร

2.12 โครงสร้างพื้นฐานเครื่องมือแม่ข่าย

โครงสร้างพื้นฐานของระบบสารสนเทศมองได้เป็นชั้นตั้งแต่ฮาร์ดแวร์ ระบบปฏิบัติการ runtime/virtualization เครื่องข่าย ไปจนถึงแอปพลิเคชันและบริการที่ผู้ใช้เรียก แต่ละชั้นอาจเป็นจุดเริ่มของปัญหาได้ เช่น disk เต็มที่ขึ้นจัดเก็บหรือ process ใช้หน่วยความจำมากในชั้นแอป เมื่ออ่าน metric ควรระบุให้ชัดว่าค่านั้นอธิบายชั้นใด

โปรแกรมนี้อ่าน host metrics ของเครื่องที่รันแอป และอ่านสถานะของ container ผ่าน Docker CLI จึงให้มุมมองบางส่วนของโครงสร้างระบบ ไม่ได้ตรวจ network traffic, latency, log ของทุก service, hardware sensor หรือระบบ cluster ตามที่ร่าง PDF เดิมเสนอไว้

2.13 การบริหารงานปฏิบัติการ IT

งาน IT Operations ครอบคลุมการตรวจสอบสถานะ การรับเหตุ การวิเคราะห์ การบรรเทาผลกระทบ และการทบทวนหลังเหตุ ระบบ monitoring เป็นส่วนหนึ่งของวงจรนี้: ข้อมูลที่เก็บช่วยระบุอาการ alert ดึงความสนใจของผู้ดูแล และการตรวจสอบ logs/สถานะบริการใช้ประกอบการแก้ไข หลังแก้แล้วควรตรวจสอบซ้ำและบันทึกสิ่งที่เกิดขึ้น

เครื่องมือขนาดเล็กช่วยรวบรวมคำสั่งซ้ำ ๆ ได้ แต่ไม่ได้แทนการกำหนดผู้รับผิดชอบ แผน incident response หรือการสำรองและกู้คืนข้อมูล การใช้ automation จึงควรมีขอบเขต action ชัด มี cooldown และมีวิธีตรวจสอบว่าผลที่สั่งเกิดขึ้นจริง

2.14 AIOps ในฐานะแนวคิดอ้างอิง

AIOps เป็นคำเรียกกลุ่มแนวทางที่นำ analytics, ข้อมูลปฏิบัติการ และบางกรณีรวม Machine Learning มาใช้สนับสนุนงาน IT Operations งานสำรวจของ Notaro, Cardoso และ Gerndt แสดงให้เห็นว่างาน AIOps ครอบคลุมหลายประเภท ตั้งแต่ resource management ไปจนถึงการตรวจจับและจัดการความล้มเหลว และยังมีวิธี/คำศัพท์หลากหลาย [11]

การใช้คำว่า Intelligent ในชื่อโครงการไม่ควรถูกตีความว่าโปรแกรมมีโมเดลเรียนรู้ครบวงจร ระบบรุ่นที่ตรวจใช้กฎ EMA และ threshold ที่กำหนดไว้ล่วงหน้า ไม่มีขั้น train หรือ model artifact กลไกนี้อธิบายได้ง่ายและพร้อมทำงานโดยไม่เตรียมข้อมูลฝึก แต่ไม่ปรับ threshold ตามพฤติกรรมเฉพาะของ server

2.15 แนวทางตรวจจับความผิดปกติที่เกี่ยวข้อง

การตรวจจับแบบ supervised ต้องใช้ตัวอย่างที่มีป้ายกำกับปกติ/ผิดปกติ ส่วน unsupervised พยายามหาโครงสร้างหรือค่าที่ต่างจากข้อมูลส่วนใหญ่โดยไม่ใช้ label และ semi-supervised มักสร้างแบบจำลองจากข้อมูลปกติแล้วตรวจตัวอย่างใหม่ วิธีที่เลือกต้องพิจารณาคุณภาพข้อมูล ค่าใช้จ่ายในการฝึก และการอธิบายผล

Isolation Forest เป็นตัวอย่างอัลกอริทึม unsupervised ที่สุ่มแบ่ง feature และสังเกตจำนวนขั้นที่ใช้แยกตัวอย่าง งานต้นฉบับของ Liu, Ting และ Zhou เสนอแนวคิดการ isolate จุดผิดปกติ ซึ่งมักถูกแยกได้เร็วกว่า instance ปกติ [12] อย่างไรก็ตาม โค้ดโครงการปัจจุบันไม่ได้ import หรือฝึก Isolation Forest; ภาพ/คะแนนจากร่าง PDF เดิมจึงไม่ใช่ผลจากระบบรุ่นนี้

2.16 การวิเคราะห์หอนุกรมเวลาและการบำรุงรักษาเชิงพยากรณ์

metrics ที่มี timestamp เป็นอนุกรมเวลา การทำ smoothing ช่วยลดความผันผวนของค่ารายรอบ NIST อธิบาย exponential smoothing ว่าให้น้ำหนักข้อมูลล่าสุดมากกว่าอดีตที่เก่าลง [9] แต่การทำ smoothing ไม่เท่ากับการทำนายระยะยาว การพยากรณ์ต้องประเมินกับข้อมูลที่กินไว้และรายงานค่าคลาดเคลื่อน

การบำรุงรักษาเชิงพยากรณ์ใช้แนวโน้มคาดการณ์เพื่อวางแผนก่อนถึงจุดวิกฤต ขณะที่โครงการนี้มีแนวโน้ม CPU จากการเปรียบเทียบค่าเฉลี่ยสองช่วงและ forecast แบบเส้นตรงจำกัดความชันใน brain.py; ยังไม่มีข้อมูลการประเมิน forecast error จึงควรเรียกว่าแนวโน้ม/ค่าประมาณใน UI ไม่ควรกล่าวว่าแม่นยำ 99.1% หรือระบุวัน disk เต็มตามตัวเลขใน PDF

2.17 มุมมองการเฝ้าระวังบริการ

Google SRE เสนอให้มองสัญญาณบริการอย่าง latency, traffic, errors และ saturation ประกอบกัน [10] ระบบนี้ครอบคลุมการใช้ทรัพยากรบางส่วนซึ่งเกี่ยวข้องกับ saturation และสถานะ container แต่ไม่ได้เก็บ latency, request traffic หรือ error rate ของแอปโดยตรง การเพิ่มข้อมูลบริการเหล่านี้เป็นแนวทางต่อยอด และต้องนิยามแหล่งข้อมูลกับช่วงวัดให้ชัด

2.18 ตัวชี้วัดสำหรับการประเมินการตรวจจับ

หากมีชุดเหตุการณ์ที่ติดป้ายกำกับ สามารถจัดผลเป็น true positive (ตรวจพบเหตุจริง), false positive (แจ้งทั้งที่ไม่มีเหตุ), true negative (ไม่แจ้งเมื่อปกติ) และ false negative (พลาดเหตุ) จากนั้นคำนวณ precision, recall และ F1-score ตามนิยามเครื่องมือประเมิน classification [13] accuracy เพียงอย่างเดียวอาจสูงได้เมื่อเหตุผิดปกติเกิดน้อยมาก จึงควรรายงาน confusion matrix และจำนวนตัวอย่างประกอบ

ตัวชี้วัดเวลา เช่น detection latency, command completion time และ MTTR ต้องเริ่มจับเวลาจากจุดเริ่มที่กำหนดเหมือนกันทุกครั้ง ส่วน availability ต้องกำหนดนิยาม service up/down และช่วงสังเกต การวัดก่อน/หลังต้องควบคุม workload และจำนวนบริการให้ใกล้เคียงกัน รายงานทดสอบที่มีอยู่ยังไม่ให้ข้อมูลสำหรับคำนวณตัวชี้วัดเหล่านี้

2.19 ขอบเขตการประยุกต์ใช้แนวคิดในระบบ

แนวคิด/ความสามารถ	สถานะในซอร์สปัจจุบัน	การอธิบายในเล่ม
Isolation Forest / LSTM	ไม่พบโมเดลหรือการฝึก	อธิบายเป็นแนวคิดที่เกี่ยวข้อง/งานต่อยอด
CPU/RAM/disk และ Docker	มีการอ่าน/ควบคุมตามโมดูล	อธิบายเป็นความสามารถของระบบ
Network, latency, logs ทุกระบบ	ไม่พบการเก็บครบตามขอบเขตเดิม	ระบุเป็นข้อจำกัด
Rollback และ MTTR ลดลง	ไม่มีหลักฐานยืนยัน	ไม่สรุปผลและเสนอให้ทดสอบ
EMA/hysteresis และ optional RAM rule	มีใน brain.py/ram_ai_manager.py	อธิบายพารามิเตอร์จริงและข้อจำกัด

2.20 สรุป

เทคโนโลยีในโครงการเชื่อมการอ่าน metrics, การประเมินด้วยกฎ, การจัดการ Docker และเว็บแดชบอร์ดเข้าด้วยกัน จุดสำคัญของการประเมินคือแยกให้ชัดระหว่างความสามารถที่พบในโค้ด ผล self-test ของแพ็คเกจ และผลจากการใช้งานกับระบบจริง

บทที่ 3 การออกแบบและพัฒนา

3.1 ภาพรวมการทำงาน

โปรแกรมเดสก์ท็อปเป็นจุดประสานงานหลัก โดยอ่านค่าทรัพยากรจากเครื่อง ส่งค่าไปยังตัวประเมินสถานะ บันทึก metrics และปรับข้อมูลที่เว็บเซิร์ฟเวอร์ใช้แสดง เมื่อผู้ใช้จัดการ container คำสั่งจะถูกส่งผ่าน Docker CLI จากโมดูลที่รับผิดชอบ การแยกโมดูลทำให้ส่วนติดต่อไม่ต้องรวมรายละเอียดการอ่านระบบ การคำนวณ และการเรียกคำสั่งไว้ในฟังก์ชันเดียว

โมดูล	หน้าที่
main.py	สร้างหน้าต่างหลัก เชื่อมการตั้งค่า การแจ้งเตือน การสำรอง และบริการเว็บ
collector.py	อ่าน CPU/RAM/disk และเขียน/อ่านข้อมูล CSV
brain.py	คำนวณ EMA, hysteresis, แนวโน้ม และค่าประเมิน risk จาก metrics
docker_ops.py	เรียก Docker CLI เพื่อดูและจัดการ container, logs, tunnel และ resource limits
remediator.py	ดำเนินการแก้ไขบางชนิดตามเงื่อนไข เช่น cache, ไฟล์ชั่วคราว, process หรือ container
server.py และ templates/	ให้บริการหน้าเว็บและ API ที่ตรวจสอบ session ก่อนสั่งงาน
backup_manager.py	สร้างไฟล์ ZIP ตามแหล่งข้อมูลและตำแหน่งปลายทางที่ตั้งไว้

3.2 ลำดับการทำงานหลัก

เมื่อเริ่มโปรแกรม จะโหลดค่ากำหนดจากไฟล์เดอร์ข้อมูลของแอป เตรียม UI และเริ่ม worker ที่จำเป็น จากนั้นการอ่าน metrics จะทำเป็นรอบ: collector อ่านค่าระบบ ☐ บันทึก CSV ☐ ส่งค่าให้ brain คำนวณ EMA และสถานะ ☐ UI อัปเดตการ์ดและกราฟ ☐ notifier แสดง popup เมื่อถึงเงื่อนไข ส่วน web server อ่าน snapshot สถานะร่วมกันและตอบคำขอจาก dashboard

ลำดับ	เหตุการณ์	ผลลัพธ์
1	เริ่มโปรแกรมและโหลดการตั้งค่า	คืนค่าเริ่มต้น รายการ YAML, limit, alert และ backup preference

ลำดับ	เหตุการณ์	ผลลัพธ์
2	อ่าน metric ของ host	ได้ CPU, RAM และ disk ของพาร์ติชันระบบ
3	ประเมินค่า	คำนวณ EMA, risk score และแนวโน้ม CPU
4	ตรวจ threshold/cooldown	คงหรือเปลี่ยนสถานะ และส่งแจ้งเตือนเมื่ออนุญาต
5	เรียก remediation (ถ้าเข้าเงื่อนไข)	ส่งคำสั่ง action พร้อม cooldown
6	ปรับหน้าจอและเว็บ	แสดงสถานะล่าสุดและรายการเหตุการณ์ของ session

3.3 รูปแบบข้อมูลและไฟล์

ข้อมูลแอปถูกแยกตามหน้าที่ CSV เก็บตัวอย่าง metrics; JSON เก็บการตั้งค่า เช่น preferences, resource limits, alert thresholds, tracked containers และบัญชีเว็บ; ส่วนไฟล์ YAML เป็นอินพุตสำหรับบริการ Docker Compose การแยกข้อมูลเช่นนี้ทำให้การตั้งค่าถูกอ่านข้ามโมดูลได้ แต่ผู้ใช้ควรสำรองและปกป้องไฟล์เหล่านี้ โดยเฉพาะไฟล์ที่มีข้อมูลบัญชีหรือค่า SMTP

ไฟล์/รูปแบบ	ตัวอย่างข้อมูล	หมายเหตุ
metrics_db.csv	timestamp, cpu, ram, disk	บันทึกต่อท้าย; ใช้แสดงประวัติ
resource_limits.json	limit CPU/RAM ต่อชื่อ container	ใช้ร่วมระหว่าง UI และโมดูล Docker
alert_thresholds.json	ค่าเตือนราย server	ไม่ใช่ threshold anomaly EMA ระดับ host
tracked_containers.json	รายการที่ import แล้ว	ใช้กรองรายการที่แสดงบนเว็บ
app_preferences.json	ธีม ขนาด UI, backup path/สถานะ	ค่าผู้ใช้
notifier_config.json	เปิด popup/cooldown	ค่าเริ่มต้น cooldown 60 วินาที
email_config.json	SMTP sender/host/port/credential	ควรจัดเก็บอย่างระมัดระวัง

3.4 การจัดการสถานะและข้อผิดพลาด

ระบบถือการเปลี่ยนสถานะเป็นสองช่วง: ปกติและผิดปกติ ค่า EMA ที่สูงสุดเป็นตัวกำหนด risk score ส่วนการแจ้งเตือนราย server ใช้ threshold ที่ตั้งเฉพาะรายการและ cooldown ของตัวเอง ข้อความ Action Log แสดงผลการดำเนินการในหน้าจอระหว่างการทำงาน ไม่ใช่ event log การที่นำกลับมา audit ได้หลังปิดโปรแกรม

การเรียก Docker มีการจัดการกรณีไม่พบคำสั่งหรือ timeout และส่งข้อความผลลัพธ์กลับมาที่ UI สำหรับการ execute ภายใน container มี timeout เพื่อไม่ให้คำสั่งค้างไม่สิ้นสุด อย่างไรก็ตามข้อความ error ไม่ได้แทนการวิเคราะห์สาเหตุ ผู้ดูแลจึงควรอ่าน container logs และตรวจสอบสภาพแวดล้อมก่อนลองซ้ำ

3.5 การเก็บและประเมิน metrics

รอบอ่านค่าระบบเรียก psutil เพื่ออ่าน CPU, หน่วยความจำ และ disk ของพาร์ทิชันระบบหลัก จากนั้นเขียนเวลาและค่าทั้งสามลง CSV การแสดงประวัติอ่านแถวล่าสุดจากไฟล์ ค่าของ container และ process บางรายการอ่านผ่าน Docker หรือข้อมูล process ของระบบ โดยขึ้นกับหน้าจอและบริการที่เลือก

ตัวประเมินเก็บ EMA แยกสำหรับ CPU, RAM และ disk พร้อมประวัติไม่เกิน 20 ค่า เมื่อได้ค่ารอบใหม่จะหา metric ที่มีค่าสูงสุดและใช้เป็น risk score จากนั้นเปลี่ยนสถานะด้วย threshold 90% และ 80% ส่วนแนวโน้ม CPU ใช้ค่าเฉลี่ยของตัวอย่างล่าสุดสามค่าเทียบกับสามค่าก่อนหน้า ข้อมูลแนวโน้มนี้เป็นการบอกทิศทางอย่างง่าย ไม่ใช่การพยากรณ์โหลดที่ผ่านการประเมินความแม่นยำ

3.6 การจัดการ Docker

ผู้ใช้เพิ่มไฟล์ Compose เพื่อให้โปรแกรมอ่านบริการและเชื่อมรายการที่เลือกกับ container ที่มีอยู่ การสั่งงานรองรับ start, stop, restart, อ่าน logs และส่งคำสั่งภายใน container โดยคำสั่งมีเวลารอสูงสุดตามที่กำหนด นอกจากนี้ผู้ใช้กำหนด CPU เป็นเปอร์เซ็นต์และ RAM เป็นเมกะไบต์ได้ โปรแกรมสั่ง docker update แล้วตรวจค่า memory limit ที่ container ใช้จริงก่อนรายงานความสำเร็จ นอกจากนี้ยังมีตัวเลือกปรับ RAM อัตโนมัติเป็นราย container ซึ่งปิดไว้จนกว่าผู้ใช้จะเปิด ระบบอ่านหน่วยความจำที่ใช้ ทำ EMA แยก container เมื่อพื้นที่ 40% บัดค่าเป็นช่วง 64 MB และเคารพเพดาน RAM ส่วนกลาง โดยจะส่งเปลี่ยนเมื่อค่าที่คำนวณต่างจาก limit เดิมอย่างน้อย 15% กลไกนี้เป็นกฎคำนวณ deterministic และยังไม่มียอดทดสอบการปรับโหนดจริงแนบมา

3.7 เว็บแดชบอร์ดและการเชื่อมต่อ

เว็บเซิร์ฟเวอร์ทำงานเบื้องหลังและให้บริการ dashboard ผ่าน LAN หากเปิดใช้ tunnel โปรแกรมติดตามสถานะ container ที่ใช้ cloudflared และอ่าน URL จาก log เพื่อแสดงบนหน้าหลัก เส้นทาง API สำหรับสถานะ container การสั่งงาน การรันคำสั่ง ping การตั้ง limit และการอ่าน log ตรวจว่ามี session เข้าสู่ระบบก่อนทำงาน ผู้ใช้ตั้งค่า SMTP แล้วส่งลิงก์ dashboard ให้ผู้รับทางอีเมลได้ การส่งอีเมลเป็นการแชร์ลิงก์ ไม่ใช่ช่องทางแจ้งเตือนเหตุผิดปกติ การทดสอบเชิงความปลอดภัยอย่างละเอียดอยู่นอกหลักฐานที่มี

3.8 การแจ้งเตือนและการดำเนินการอัตโนมัติ

เมื่อสถานะเข้าเกณฑ์ผิดปกติ โปรแกรมสามารถแสดง popup และเรียก remediation ที่สัมพันธ์กับเงื่อนไข การแจ้งเตือนทรัพยากรราย server มี cooldown 30 วินาที, popup anomaly มีค่าเริ่มต้น 60 วินาที และ remediation มี cooldown 15 วินาที กลไกเหล่านี้ช่วยจำกัดการเรียกซ้ำ แต่เป็นคนละช่องทางและคนละการตั้งค่า การกระทำกับ process หรือไฟล์อาจส่งผลกระทบต่อบริการ จึงควรตรวจสอบรายการและทดสอบในสภาพแวดล้อมจำลอง ก่อนเปิดใช้กับเครื่องสำคัญ

3.9 การสำรองข้อมูล

เมื่อผู้ใช้เปิดระบบสำรองและกำหนดปลายทาง scheduler จะทำรอบแรกทันทีและทำซ้ำทุกหนึ่งชั่วโมง แหล่งข้อมูลมาจากรายการที่โปรแกรมรวบรวม เช่น โฟลเดอร์ server/โครงการที่เพิ่มไว้และไฟล์ตั้งค่าที่เกี่ยวข้อง ไฟล์ ZIP ใหม่จะเขียนเป็นไฟล์ชั่วคราวก่อน เมื่อสร้างเสร็จจึงลบชุดเก่าและเปลี่ยนชื่อไฟล์ ระบบจึงเก็บชุดล่าสุดเป็นหลัก และไม่มีหลักฐานการทดสอบ restore ในชุดข้อมูลปัจจุบัน

3.10 เครื่องมือและสภาพแวดล้อมที่ยืนยันได้

ซอร์สโค้ดใช้ Python, CustomTkinter, psutil, pandas, Flask, Docker CLI และ YAML สำหรับอ่าน Compose; ชุดส่งมอบมี Windows EXE แบบ one-file ผล self-test ที่บันทึกไว้ระบุว่าทดสอบบน Windows วันที่ 23 กันยายน 2569 แต่ไม่ได้ระบุรุ่น Windows หรือสเปกฮาร์ดแวร์ จึงไม่ขยายผลไปยังสภาพแวดล้อมอื่น

3.11 การสร้างไฟล์ส่งมอบ

กระบวนการ release ใช้ PyInstaller สร้าง Windows one-file executable โดยระบุโมดูลและข้อมูลที่ต้องฝัง เช่น templates, theme, logo และ Compose resource หนึ่งไฟล์ช่วยให้แจกจ่ายง่าย แต่ข้อมูลที่ผู้ใช้แก้ไขต้องเขียนลง app-data ที่ถาวร ไม่ควรเขียนกลับเข้าไปใน resource ที่ถูกแตกชั่วคราวระหว่างรัน [6] โครงการมี self-test เพื่อเช็ค Tcl/Tk, resources, themes, YAML, Flask test client, app-data และ relaunch

3.12 วิธีประเมินผล

รายงานผลแยกเป็นสองส่วน ส่วนแรกคือรายการทดสอบการทำงานที่แนบมากับโครงการ โดยรายงานแหล่งที่มาและขอบเขตตามเอกสาร ส่วนที่สองคือ self-test ของ EXE ซึ่งตรวจการบรรจุทรัพยากรและการเริ่มโปรแกรม การทดสอบทั้งสองกลุ่มไม่มีข้อมูลดิบเพียงพอสำหรับคำนวณความแม่นยำ เวลาในการตอบสนอง หรือความพร้อมใช้งานระยะยาว

บทที่ 4 ผลการพัฒนาและการทดสอบ

4.1 ผลการพัฒนา

ผลการพัฒนาเป็นโปรแกรมเดสก์ท็อปที่รวมหน้าต่างติดตามทรัพยากร ประวัติกราฟ การจัดการ container การตั้งค่า resource limit การแจ้งเตือน และเมนูเปิดเว็บแดชบอร์ดไว้ในระบบเดียว ซอร์สยังมีโมดูลสำหรับสำรองข้อมูลและเตรียม Docker Compose ที่จำเป็นสำหรับบริการเว็บ/tunnel ใน EXE

การตรวจสอบซอร์สยืนยันว่า metrics ของเครื่องเขียนลง CSV ส่วน Action Log และรายการ web events ใช้แสดงเหตุการณ์ในระหว่าง session ไม่พบการเขียน event log เหล่านี้เป็นไฟล์ถาวรในส่วนที่ตรวจ การสำรองสร้าง ZIP ตามแหล่งข้อมูลที่กำหนดและเก็บชุดล่าสุด จึงควรระบุขอบเขตนี้ทุกครั้งเมื่ออธิบายความสามารถ

4.2 หน้าจอของโปรแกรม

ภาพต่อไปนี้เป็นภาพประกอบหน้าจอที่อยู่ในรายงานทดสอบซึ่งส่งมาพร้อมโครงการ ภาพช่วยอธิบายตำแหน่งเมนูและรูปแบบหน้าจอ แต่ไม่ใช่หลักฐานว่าทุกปุ่มผ่านการทดสอบในรุ่นปัจจุบัน ภาพบางหน้าไม่มีข้อมูลรายการจริง และภาพ dashboard มี IP ตัวอย่าง จึงไม่ควรใช้ค่านั้นเชื่อมต่อ

รูปที่ 4.1 หน้าหลักสำหรับติดตามทรัพยากร

ที่มา: ภาพหน้าจอในเอกสารผลทดสอบที่ผู้จัดทำส่งมา

รูปที่ 4.2 หน้าประวัติและกราฟการใช้ทรัพยากร

หน้าจอแสดงแนวคิดการเรียกดูข้อมูลย้อนหลัง; ข้อมูลบนภาพไม่ใช่ผลวัด benchmark

รูปที่ 4.3 หน้า Server Dashboard

ภาพประกอบจากรายงานเดิม; ตรวจรายการและที่อยู่ตัวอย่างก่อนนำไปใช้เป็นหลักฐานฉบับส

ง

รูปที่ 4.4 หน้า Server Manager

ภาพแสดงพื้นที่จัดการรายการ server/container

รูปที่ 4.5 หน้าต่าง Terminal

การส่งคำสั่งทำงานภายใน container ที่เลือก; ควรใช้งานด้วยความระมัดระวัง

รูปที่ 4.6 หน้า Action Log

รายการเหตุการณ์เป็นข้อมูลระหว่าง session ตามการตรวจสอบข้อสงสัยที่มี

4.3 รายงานผลทดสอบเชิงหน้าที่

เอกสารผลทดสอบที่ส่งมาพร้อมโครงการระบุการทดสอบแบบ black-box 10 รายการและรายงานว่าผ่านทั้งหมด รายงานต้นทางระบุว่าดำเนินการโดย Claude บนเครื่องที่ติดตั้ง Docker Desktop แต่ไม่ได้แนบรายละเอียดวันทดสอบ รุ่นระบบปฏิบัติการ สเปกเครื่อง log ดิบ หรือหลักฐานแยกในแต่ละกรณี ตารางจึงแสดงผลตามเอกสารนั้น ไม่ใช้การทดสอบซ้ำโดยคณะผู้จัดทำ

รายการ	ผลที่ระบุในรายงาน	ขอบเขตการอ่านผล
อ่าน CPU/RAM/disk	ผ่าน	ไม่มีค่าจริงหรือตารางเปรียบเทียบแนบ
ตรวจ anomaly และ hysteresis	ผ่าน	ไม่มีลำดับค่าที่ป้อนหรือเวลาที่แจ้งเตือน
เริ่ม/หยุด/เริ่มใหม่ container	ผ่าน	ไม่มีชื่อ/รุ่น container ในหลักฐาน
เว็บ login/dashboard ใน LAN	ผ่าน	รายงานไม่ได้แยก first-run และ API ทุกเส้นทาง
Cloudflare Quick Tunnel	ผ่าน	ไม่มีรายละเอียดเครือข่ายหรือ URL ใช้ซ้ำได้
สร้าง ZIP backup	ผ่าน	ยืนยันการสร้างไฟล์ ไม่ได้รายงานการ restore
กราฟประวัติและคำสั่ง terminal	ผ่าน	เป็นผลตามรายการทดสอบที่รายงาน
ธีมและขนาด UI	ผ่านในเอกสารต้นทาง	เอกสารกล่าวถึง Light แต่รุ่นปัจจุบันรองรับ Dark/Gray; ต้องทดสอบซ้ำสำหรับรุ่นปัจจุบัน

รหัส	กรณีทดสอบ	ผลตามรายงาน	ข้อสังเกต
B01	อ่าน CPU/RAM/disk	ผ่าน	ไม่มีตัวเลขเทียบหรือ log แนบ
B02	ตรวจ EMA เมื่อเกินเกณฑ์	ผ่าน	ไม่มี input trace

รหัส	กรณีทดสอบ	ผลตามรายงาน	ข้อสังเกต
B03	Hysteresis ระหว่างแกว่งไกว threshold	ผ่าน	ไม่มีลำดับค่าทดสอบ
B04	Start/stop/restart container	ผ่าน	ไม่ระบุ image หรือ container ที่ใช้
B05	Login และ dashboard ผ่าน LAN	ผ่าน	ไม่ครอบคลุมทุก API
B06	เปิด Cloudflare Quick Tunnel	ผ่าน	ไม่มีผลทดสอบซ้ำจากเครือข่ายภายนอกแนบ
B07	สร้าง ZIP backup	ผ่าน	ไม่มีผล restore
B08	เปลี่ยน theme/ขนาด UI	ผ่านในเอกสารเดิม	ผล Light ไม่ตรงกับชุดรุ่นปัจจุบัน
B09	บันทึกและดูกราฟประวัติ	ผ่าน	ไม่มี CSV ตัวอย่างแนบ
B10	ส่งคำสั่งผ่าน terminal ใน container	ผ่าน	ไม่แนบคำสั่งและผลลัพธ์รายการนี้

B01–B10 เป็นการถอดรายการจากเอกสารทดสอบต้นทางที่ผู้จัดทำส่งมา ผู้ทำรายงานฉบับนี้ไม่ได้ถือว่าตนเองรันทดสอบเหล่านี้ซ้ำ การนำผลไปอ้างอิงควรแนบต้นฉบับหรือขอให้ผู้ทดสอบยืนยันรายละเอียดเพิ่ม

4.4 ผล self-test ของ Windows EXE

รายงาน self-test ระบุว่าเรียก Intelligent-Server-Operations-Platform.exe ด้วยอาร์กิวเมนต์ --self-test-relaunch บน Windows วันที่ 23 กันยายน 2569 และผ่าน 7 รายการ ได้แก่ Tcl/Tk, ทรัพยากรที่ฝังใน EXE, ธีม Dark/Gray, การอ่าน Docker Compose YAML, เว็บ setup/login ผ่าน Flask test client, การเขียนอ่าน app-data และการเปิด EXE ซ้ำแบบ one-file relaunch

การทดสอบนี้ตรวจองค์ประกอบ runtime และแพ็คเกจ ไม่ได้วัดประสิทธิภาพบนเครื่องแม่ข่าย ไม่ได้ควบคุม container จริง ไม่ได้ทดสอบ Cloudflare จากเครือข่ายภายนอก และไม่ใช้หลักฐานการเปิดโปรแกรมต่อเนื่องระยะยาว

4.5 อภิปรายผล

หลักฐานที่มีแสดงว่าโปรแกรมประกอบส่วนต่าง ๆ เข้าด้วยกันได้ และรายงานเชิงหน้าที่ระบุว่ากรณีที่ทดลองผ่าน ส่วน self-test ช่วยยืนยันว่าการบรรจุเป็น EXE ยังโหลดส่วนประกอบสำคัญได้ อย่างไรก็ตาม ข้อมูลที่มีไม่พอจะสรุปว่าระบบตรวจจับได้แม่นยำเพียงใด หรือทำงานได้เหมือนกันบนเครื่องทุกแบบ

การตั้ง threshold 90/80 และ alpha 0.3 เป็นค่าคงที่ช่วยให้พฤติกรรมตรวจสอบได้ แต่ยังอาจไม่เหมาะสมกับบริการที่ใช้ทรัพยากรสูงตามปกติ ส่วน backup ยังไม่มีผลกู้คืน จึงสรุปได้เพียงว่ามีการสร้าง ZIP ตามรายงาน ไม่อาจยืนยันความครบถ้วนของข้อมูลหลัง restore

4.6 ข้อจำกัดของผลทดสอบ

- ไม่ระบุสเปกเครื่องและรุ่นซอฟต์แวร์ที่ใช้ใน black-box test
- ไม่มีชุดข้อมูล metrics และเวลาตอบสนองแยกตามกรณี
- ไม่มีหลักฐานการทดสอบ restore, cooldown, การเปิดต่อเนื่อง 8 ชั่วโมง และการเข้าถึง API โดยไม่ login
- ผลสี Light ในรายงานเดิมไม่ตรงกับรุ่นปัจจุบันที่เหลือ Dark/Gray
- ไม่พบการวัดก่อนและหลังเพื่อสรุปการลด downtime หรือภาระงานผู้ดูแล

บทที่ 5 สรุปผลและข้อเสนอแนะ

5.1 สรุปผล

โครงการพัฒนาเครื่องมือสำหรับดูการใช้ CPU, RAM และ disk ของเครื่องที่เปิดโปรแกรม พร้อมจัดการ Docker container ที่เลือกจาก Docker Compose ระบบใช้ EMA และ threshold แบบ hysteresis เพื่อแสดงสถานะตามกฎ มีเว็บแดชบอร์ดที่ต้องเข้าสู่ระบบ บันทึก metrics ลง CSV และสำรองข้อมูลที่กำหนดเป็น ZIP โปรแกรมรุ่นส่งมอบถูกจัดเป็น Windows EXE แบบ one-file และมี self-test สำหรับตรวจ runtime

หลักฐานที่แนบมาสรุปผล black-box 10 รายการและ self-test 7 รายการว่าผ่านตามขอบเขตของแต่ละรายงาน ข้อมูลยังไม่เพียงพอสำหรับสรุปประสิทธิภาพเชิงปริมาณ ความทนทานระยะยาว ความแม่นยำในการตรวจจับ หรือผลต่อ downtime

5.2 ข้อจำกัดของระบบ

- เกณฑ์และน้ำหนัก EMA ยังเป็นค่าคงที่ ไม่ได้ปรับตามชนิดงานหรือ workload
- disk ที่อ่านเป็นพาร์ทิชันระบบหลัก และการจัดการ container ต้องมี Docker ใช้งานได้
- Quick Tunnel เป็นช่องทางสำหรับทดลอง การเปิดใช้จริงจากภายนอกต้องจัดการความปลอดภัยและรูปแบบ tunnel ให้เหมาะสม
- ระบบสำรองเก็บ ZIP ชุดล่าสุดตามรายการแหล่งข้อมูล ไม่รวม host และ Docker volumes โดยอัตโนมัติ และยังไม่มีผลทดสอบกู้คืน
- การแก้ไข process หรือไฟล์อัตโนมัติอาจมีผลต่อบริการ;
ควรจำกัดสิทธิ์และทดสอบก่อนใช้งานจริง
- ยังไม่พบการจัดเก็บ Action Log/web events ถาวรในโค้ดที่ตรวจ

5.3 ข้อเสนอแนะสำหรับการพัฒนาต่อ

- ทำชุดทดสอบซ้ำได้ โดยบันทึกสภาพแวดล้อม input, ผลที่คาดหวัง, ผลจริง และเวลาที่ใช้ในแต่ละกรณี
- ทดสอบ backup ด้วยการ restore ไปยังตำแหน่งแยก และตรวจไฟล์สำคัญหลังการกู้คืน

- เพิ่มตัวเลือกเกณฑ์รายเครื่องหรือราย container พร้อมรายการยกเว้นสำหรับ process ที่ห้ามจัดการ

- เพิ่มการเก็บ event log แบบถาวรโดยหลีกเลี่ยงรหัสผ่าน token และข้อมูลอ่อนไหว

- ทบทวนการยืนยันตัวตน สิทธิ์ API การจัดการ secret และการเชื่อมต่อ HTTPS ก่อนเปิดใช้ภายนอก

- ประเมินค่า EMA และ threshold ด้วย workload หลากรูปแบบ ก่อนกล่าวถึงความแม่นยำหรือการลดเวลาหยุดให้บริการ

5.4 งานที่ควรทำก่อนส่งฉบับสมบูรณ์

ให้ผู้จัดทำตรวจชื่อเรื่อง ชื่อบุคคล และรูปแบบชื่อหลักสูตรตามเอกสารทางการ
เพิ่มหลักฐานจากการทดสอบรุ่นล่าสุด และใช้แบบฟอร์มอนุมัติของหลักสูตรก่อนพิมพ์ฉบับส่ง
ส่วนข้อมูลส่วนตัวที่ไม่จำเป็นต่อการประเมินไม่ควรใส่ในเล่ม

บรรณานุกรม

- [1] psutil developers. (ม.ป.ป.). psutil documentation. สืบค้นเมื่อ 26 กันยายน 2569, จาก <https://psutil.readthedocs.io/stable/>
- [2] Docker, Inc. (ม.ป.ป.). Resource constraints. Docker Documentation. สืบค้นเมื่อ 26 กันยายน 2569, จาก https://docs.docker.com/engine/containers/resource_constraints/
- [3] Pallets Projects. (ม.ป.ป.). Quickstart — Flask Documentation. สืบค้นเมื่อ 26 กันยายน 2569, จาก <https://flask.palletsprojects.com/en/stable/quickstart/>
- [4] Cloudflare. (2026, 20 เมษายน). Quick Tunnels. Cloudflare One Documentation. <https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/do-more-with-tunnels/trycloudflare/>
- [5] Python Software Foundation. (ม.ป.ป.). csv — CSV File Reading and Writing. Python Documentation. <https://docs.python.org/3/library/csv.html>
- [6] PyInstaller Development Team. (ม.ป.ป.). What PyInstaller Does and How It Does It. PyInstaller Documentation. <https://pyinstaller.org/en/stable/operating-mode.html>
- [7] Tom Schimansky. (ม.ป.ป.). Documentation Introduction. CustomTkinter Documentation. <https://customtkinter.tomschimansky.com/documentation/>
- [8] pandas development team. (ม.ป.ป.). pandas.read_csv. pandas documentation. https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html
- [9] NIST/SEMATECH. (ม.ป.ป.). What is Exponential Smoothing? e-Handbook of Statistical Methods. <https://www.itl.nist.gov/div898/handbook/pmc/section4/pmc43.htm>
- [10] Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (บ.ก.). (2016). Monitoring Distributed Systems. ใน Site Reliability Engineering. Google. <https://sre.google/sre-book/monitoring-distributed-systems/>

[11] Notaro, P., Cardoso, J., & Gerndt, M. (2020). A Systematic Mapping Study in AIOps. arXiv:2012.09108. <https://arxiv.org/abs/2012.09108>

[12] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. 2008 Eighth IEEE International Conference on Data Mining. <https://doi.org/10.1109/ICDM.2008.17>

[13] scikit-learn developers. (ม.ป.ป.). Model evaluation: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html

หมายเหตุ: บรรณานุกรมนี้ครอบคลุมเอกสารทางเทคนิคที่ใช้อธิบายเครื่องมือในระบบ
ควรเพิ่มงานวิจัยที่เกี่ยวข้องตามที่อาจารย์ที่ปรึกษาหรือคู่มือหลักสูตรกำหนด

ภาคผนวก ก

คู่มือการใช้งานระบบปฏิบัติการและบริหารจัดการเครื่องแม่ข่าย

แนวทางการใช้งาน

1. เตรียมเครื่อง

ติดตั้งและเปิด Docker Engine/Desktop หากต้องใช้การจัดการ container ตรวจสอบว่า Docker daemon พร้อมใช้งานและบัญชีผู้ใช้มีสิทธิ์เรียก Docker CLI

2. เปิดโปรแกรม

เปิดไฟล์ EXE จากนั้นตรวจหน้า dashboard ว่าค่า CPU, RAM และ disk ปรากฏตามเครื่องที่กำลังใช้งาน

3. เพิ่มบริการ

เลือกไฟล์ Docker Compose YAML ที่ต้องการให้โปรแกรมอ่าน ตรวจสอบชื่อบริการและ container ก่อนสั่งงาน

4. จัดการ container

ตรวจสอบสถานะก่อน แล้วเลือก start, stop หรือ restart ตามความจำเป็น คำสั่ง terminal จะทำงานภายใน container ที่เลือก

5. ใช้เว็บแดชบอร์ด

เปิดเว็บจากลิงก์ LAN หรือ Quick Tunnel ที่โปรแกรมแสดง
เข้าสู่ระบบก่อนดูสถานะหรือส่งคำสั่ง

6. สำรองข้อมูล

เปิดการสำรอง เลือกตำแหน่งปลายทางที่เข้าถึงได้ ระบบทำไฟล์ ZIP ตามรอบที่ตั้งไว้และเก็บชุดล่าสุด ก่อนพึ่งพาข้อมูลสำรองควรทดลองแตกไฟล์และกู้คืนในโพลเดอร์ทดสอบ

ภาคผนวก ข

คำสั่งแต่งตั้งคณะกรรมการจัดทำร่างคู่มือการพิมพ์ปริญาณิพนธ์

คณะครุศาสตร์อุตสาหกรรม

ภาคผนวก ค

มติคณะกรรมการประจำคณะครุศาสตร์อุตสาหกรรม

ประวัติผู้เขียน

วัชรปกรณ มีชัย

ปรัชญาพงษ์ คุณมี